

no java before code

Yes bro 👍 I'll remember this format for future explanations.

Now continuing from where we stopped.

♦ RUN CODE EVERY 4 SECONDS

```
setInterval(() => {
```

This line is extremely important.

What is setInterval()?

```
setInterval()
```

→ predefined browser function

Used for:

```
Run code repeatedly after a fixed time interval
```

Example:

```
setInterval(  
  someCode,  
  1000  
)
```

means:

```
Run someCode every 1000 milliseconds
```

What is 1000 milliseconds?

```
1000 ms = 1 second
```

So:

```
4000 ms = 4 seconds
```

What is () => ?

```
() =>
```

This is called:

Arrow Function

Think:

```
function(){  
}
```

but shorter.

Example:

Long version:

```
function(){  
  console.log("Hello");  
}
```



Arrow version:

```
() => {  
  console.log("Hello");  
}
```

Both do the same thing.

Final Meaning

```
setInterval(() => {
```

means:

Every 4 seconds

Run the code inside these braces

♦ GET EMAIL TEXT

```
let text = getEmailText();
```



What happens?

Earlier you created:

```
function getEmailText()
```

which returns:

```
email content
```

Now:

```
getEmailText()
```

calls that function.

Suppose Gmail email says:

```
Congratulations!
You won $1000
```

Then:

```
getEmailText()
```

returns:

```
"Congratulations!
You won $1000"
```

Stored into:

```
text
```

So:

```
text
```

now contains:

```
email body text
```

♦ CHECK IF EMAIL EXISTS

```
if (!text) return;
```

Many beginners find this confusing.

What is if?

```
if
```

keyword

Used for conditional logic.

What is !

```
!
```

means:

```
NOT
```

Reverse boolean value.

Example:

```
true
```

becomes

```
false
```

Example:

```
!true
```

↓

```
false
```

Example:

```
!false
```

↓

```
true
```

Why use !text ?

Suppose:

```
text = ""
```

Empty string.

That means:

```
No email found
```

Then:

```
!text
```

becomes:

```
true
```

Final Meaning

```
if (!text) return;
```

means:

```
If email text is empty
```

```
Stop immediately
```

```
Do not continue
```

Why?

Because no email exists to analyze.

Without this check:

```
fetch(...)
```

would send empty data.

- ◆ SEND REQUEST TO SPRING BOOT

```
fetch("http://localhost:8080/email/analyze", {
```

This is where your extension contacts Spring Boot.

What is fetch()?

```
fetch()
```

→ predefined browser function 

Used for:

```
Sending HTTP requests
```

Think:

```
Browser
↓
fetch()
↓
Server
```

What is this URL?

```
http://localhost:8080/email/analyze
```

Break it down.

```
http://
```

Protocol

```
localhost
```

Your own computer

Same machine

```
8080
```

Port number

Spring Boot usually runs here.

```
/email/analyze
```

API endpoint

Mapped in Spring Controller.

Example:

```
@PostMapping("/email/analyze")
```

matches:

```
/email/analyze
```

Final Meaning

```
fetch("http://localhost:8080/email/analyze", {
```

means:

Send HTTP request
to Spring Boot
at endpoint
/email/analyze

◆ REQUEST METHOD

```
method: "POST",
```

What is method?

Request type.

Common methods:

```
GET
POST
PUT
DELETE
```

Your project uses:

```
POST
```

because:

```
You are sending email data
```

to server.

Final Meaning

```
method: "POST"
```

means:

```
Send data to server
```

◆ REQUEST HEADERS

```
headers: {
  "Content-Type": "application/json"
}
```

This tells Spring Boot:

```
The data I am sending is JSON
```

Without this:

Spring may not understand format properly.

Example JSON:

```
{  
  "emailText":"hello"  
}
```

Content-Type tells server:

```
Interpret body as JSON
```
